

NEURONWORKS / LEARNING SERIES

INTERMEDIATE PROGRAMMER SQL

Correct, Explainable, Concurrent-Safe,
and Performance-Aware Data Access

TECHNICAL LEARNING BOOK — V2

PT Neuronworks Indonesia

Program Pengembangan Kompetensi Programmer | 2026

Edisi 2.0 / Agustus 2026

Panduan Penggunaan Modul

Posisi modul: Modul ini menguji kemampuan programmer menjelaskan hasil query, menjaga integritas data, membaca evidence database, dan memilih optimasi berdasarkan workload. Fokusnya bukan hafalan sintaks atau sekadar query berhasil dijalankan.

Komponen	Ketentuan
Level	Intermediate Programmer
Durasi	1 minggu; estimasi 12–16 jam self-learning di luar mentoring dan project
Prasyarat	SELECT, JOIN, DML, constraint, SQL client, Git, dan pemahaman schema aplikasi
DBMS	Gunakan DBMS project. Contoh inti dibuat portabel; bagian khusus dialek diberi label
Metode	Prediksi hasil → tulis query → uji edge case → baca plan/log → ubah satu hal → ukur ulang
Bukti	SQL script, fixture, result set, transaction test, execution plan, metric before–after, dan decision record

Cara belajar

- Nyatakan pertanyaan bisnis dan grain hasil sebelum menulis query.
- Buat dataset kecil yang memuat duplicate, NULL, missing relation, dan nilai batas.
- Uji correctness lebih dahulu; optimasi tidak boleh mengubah semantic hasil.
- Gunakan plan, row estimate/actual, I/O, lock, dan log sebagai evidence.
- Catat dialek, versi DBMS, schema, index, parameter, serta kondisi cache saat membandingkan hasil.

Batas kedalaman dan dialek

Istilah clustered/nonclustered dan sintaks PIVOT pada silabus paling dekat dengan SQL Server. PostgreSQL dan MySQL memiliki organisasi fisik serta fitur yang berbeda. Karena itu, konsep dijelaskan lintas DBMS, sedangkan contoh vendor-specific selalu diberi label. Peserta harus mengonfirmasi syntax dan behavior pada dokumentasi versi DBMS project.

Peta Pembelajaran V2

CARA MEMBACA MODUL		
Jalur Core memuat seluruh 21 topik SQL pada silabus Intermediate. Supporting memperkuat correctness, concurrency safety, observability, dan performance evidence. Reference mengikuti dialek serta versi DBMS project; sintaks vendor-specific bukan target hafalan lintas platform.		
Label	Makna	Perlakuan belajar
CORE	Langsung berasal dari silabus Intermediate.	Wajib dipahami, dipraktikkan, dan dapat dijelaskan.
SUPPORTING	Menjaga query benar, aman, observable, dan efisien.	Pelajari melalui failure mode serta evidence project.
REFERENCE	Sintaks dan fitur yang bergantung DBMS/version.	Gunakan dokumentasi resmi sesuai engine project.

Learning Path Satu Minggu

Hari	Fokus	Bab	Evidence minimum
1	Workload, page, access path, dan index	1–2	Workload/grain, plan, I/O, index evidence
2	Reports, hierarchy, pivot, dan subquery	3–5	Reconciliation, edge-case, rewrite comparison
3	Set semantics dan transaction safety	6–7	NULL matrix, concurrency, rollback/retry proof
4	Database objects, trigger, dan workload control	8–10	Dependency, multi-row, access/profile evidence
5	Tuning dan integrated refactoring	11–12	Plan diagnosis, baseline, evidence pack, defense

Bukti minimum peserta

- Schema, seed dataset, DBMS/version, dan instruksi setup yang dapat direproduksi.
- Query beserta expected result untuk normal, duplicate, NULL, empty set, missing relation, dan boundary case.
- Execution evidence: actual/estimated plan, row count, logical reads, runtime, wait/lock, atau query count.
- Integrity evidence: transaction, isolation, rollback, retry/idempotency, permission, serta multi-row behavior.
- Decision record yang menjelaskan hypothesis, alternatif, semantic risk, trade-off, hasil before-after, dan rollback.

Ringkasan Traceability

Kelompok silabus	Topik	Bab	Bukti utama
Indexing	1	1–2	Page/access path, plan, I/O, index decision
More subquery	10	3–6	Grain, reconciliation, rewrite, NULL semantics
Transaction	1	7	Concurrency, rollback, retry/idempotency
Database Objects	4	8	Dependency, permission, deterministic result, plan
Trigger	1	9	Multi-row, rollback, recursion, latency evidence
User Access Control	2	10	Workload, privilege, sanitized profiling/log
Tuning	1	11	Plan diagnosis dan before-after metric
Query Refactoring	1	12	Query count, equivalence, resource evidence

Catatan normalisasi silabus

NORMALISASI

Istilah pada sumber seperti 'NOTEXISTS', 'effect it', dan 'Pivots' ditampilkan dengan ejaan teknis yang lebih lazim tanpa mengubah urutan maupun cakupan 21 topik SQL. Clustered/non-clustered dan PIVOT tetap dibaca sesuai dialek DBMS yang digunakan project.

Capaian Pembelajaran

- menjelaskan page, access path, clustered/nonclustered concept, composite/covering index, dan write cost;
- menulis laporan kompleks dengan aggregate, self-join, pivot/conditional aggregation, dan struktur yang transparan;
- memilih subquery, EXISTS, IN, ANY, ALL, dan join dengan memahami NULL serta cardinality;
- merancang transaction boundary, isolation, retry, dan failure propagation secara aman;
- menggunakan view, layered view, top-N view, stored procedure, serta trigger secara proporsional;
- membedakan OLTP dan OLAP serta menghindari reporting workload yang merusak layanan transaksi;
- menerapkan least privilege dan membaca log/profiling tanpa mengekspos data sensitif;
- membaca execution plan, menguji hypothesis, dan mengoptimasi query dengan evidence; dan
- mendeteksi N+1 serta memperbaikinya melalui set-based query, eager loading, join, batching, atau index yang tepat.

Peta Kompetensi

Area	Kemampuan inti	Bukti
Storage & index	Page, access path, selectivity, composite/covering	Plan dan write/read trade-off
Query composition	Aggregate, self-join, pivot, subquery	Result set dan edge-case test
Concurrency	Transaction, isolation, deadlock/retry	Multi-session scenario
Database objects	View, procedure, trigger	Dependency dan side-effect record
Operations	OLTP/OLAP, access, log, profiling	Least privilege dan evidence
Tuning	Execution plan, N+1, refactoring	Before–after measurement

Isi Buku

- 01** Workload, pages, dan access paths
 - 02** Index architecture dan design decisions
 - 03** Aggregate reports yang benar dan transparan
 - 04** Self-join, hierarchy, dan pivot
 - 05** Subquery dan correlated reasoning
 - 06** EXISTS, IN, ANY, ALL, dan NULL semantics
 - 07** Transactions, isolation, dan failure handling
 - 08** Views, layered views, dan stored procedures
 - 09** Triggers: power, risk, dan alternatives
 - 10** OLTP/OLAP, access control, log, dan profiling
 - 11** Execution plans dan evidence-based tuning
 - 12** N+1 refactoring dan mini-project
-

BAB 01

Workload, pages, dan access paths

Memahami bagaimana workload, page, row, dan access path membentuk biaya query.

Klasifikasi	CORE
Fokus silabus	Indexing — cara kerja index, page layout, locality, dan pilihan access path.
Project case	Query mengembalikan sedikit baris tetapi membaca banyak page karena grain, predicate, dan access path tidak selaras dengan workload.
Bukti Intermediate	Workload statement, grain hasil, logical/physical reads, access path, row estimate/actual, serta dataset batas.
Best practice	Mulai dari pola akses dan grain bisnis; gunakan index sebagai struktur data yang memiliki read benefit sekaligus write dan storage cost.

TARGET KOMPETENSI

- Menetapkan grain dan workload sebelum tuning.
- Menjelaskan page-oriented storage secara konseptual.
- Membedakan scan, seek/search, lookup, dan sort.

1.1 Mulai dari workload dan grain

Query yang baik menjawab kebutuhan pada grain yang jelas: satu row per project, per owner-bulan, atau per status-hari. OLTP mengutamakan transaksi pendek dan konsisten; analitik mengutamakan scan, agregasi, dan sejarah. Index yang baik untuk satu workload belum tentu baik untuk workload lain.

1.2 Page dan locality

Database umumnya membaca data dalam block/page, bukan satu nilai secara terpisah. Row width, page density, locality, fragmentation, cache, dan jumlah page yang disentuh memengaruhi I/O. Istilah dan ukuran page berbeda menurut DBMS; validasi pada engine project.

1.3 Access path

Optimizer memilih access path berdasarkan predicate, statistik, cardinality, ordering, dan biaya. Scan tidak otomatis buruk: untuk sebagian besar table, scan dapat lebih murah daripada ribuan random lookup. Fokus pada row yang diperlukan, page yang disentuh, dan pekerjaan tambahan seperti sort/hash.

Contoh Bab 1. Query terarah

```
SELECT project_id, status, updated_at
FROM project_events
WHERE tenant_id = :tenant_id
  AND updated_at >= :from_date
ORDER BY updated_at DESC;
```

RED FLAG — KESALAHAN UMUM

- Tuning sebelum grain hasil jelas.
- Menganggap scan selalu buruk.
- Membandingkan query pada data terlalu kecil.
- Mengabaikan row width dan kolom yang tidak diperlukan.

PRAKTIK & EVIDENCE 1

Definisikan grain tiga laporan project, ukur jumlah row/page yang diproses, lalu jelaskan access path yang diharapkan sebelum melihat plan.

Definition of done: Grain dan workload terdokumentasi; hasil edge case benar; access path prediction dibandingkan dengan plan aktual.

CHECKPOINT Bab 1

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Mengapa database membaca page, bukan hanya nilai?

Apakah table scan selalu buruk?

Apa fungsi grain pada desain query?

BAB 02

Index architecture dan design decisions

Mendesain index sebagai struktur akses dengan manfaat baca dan biaya tulis yang terukur.

Klasifikasi	CORE
Fokus silabus	Indexing — clustered/non-clustered concept serta keputusan composite, covering, selectivity, dan sargability.
Project case	Composite index ada tetapi urutan kolom tidak mendukung predicate utama; covering index justru memperberat write tanpa manfaat terukur.
Bukti Intermediate	Index definition, plan sebelum-sesudah, read/write metric, selectivity, index usage, dan rollback decision.
Best practice	Rancang index dari workload representatif; buktikan manfaat dan biaya, hindari index spekulatif atau duplikatif.

TARGET KOMPETENSI

- Membedakan clustered dan nonclustered sebagai konsep vendor-specific.
- Memilih composite key order dan covering columns.
- Menghindari over-indexing.

2.1 Clustered dan nonclustered

Pada SQL Server, clustered index mengatur row data pada leaf level dan sebuah table hanya dapat memiliki satu clustered index; nonclustered index adalah struktur terpisah yang menunjuk row. DBMS lain tidak selalu menggunakan model atau istilah yang sama. Jangan memindahkan aturan SQL Server secara mentah ke PostgreSQL/MySQL.

2.2 Composite dan covering index

Urutan key harus mengikuti predicate equality, range, join, dan ordering yang dominan. Prefix kiri menentukan pola akses yang dapat memanfaatkan index. Included/covering columns dapat mengurangi lookup, tetapi memperbesar index dan biaya write.

Contoh Bab 2. Query terarah

```
-- SQL Server example
CREATE INDEX IX_events_tenant_date
ON dbo.project_events (tenant_id, occurred_at DESC)
INCLUDE (project_id, event_type);
-- Verify with representative parameters and actual plan.
```

2.3 Selectivity, sargability, dan write cost

Predicate pada indexed column perlu dapat dipakai sebagai search argument. Function/cast pada column, implicit conversion, atau wildcard prefix dapat menghalangi access path tertentu. Setiap index menambah storage, maintenance, dan DML cost; keputusan harus berbasis workload.

Contoh Bab 2. Query terarah

```
-- Less sargable
WHERE CAST(occurred_at AS DATE) = :day

-- Range predicate
WHERE occurred_at >= :day_start
AND occurred_at < :next_day
```

RED FLAG — KESALAHAN UMUM

- Membuat index pada setiap kolom.
- Mengabaikan urutan composite key.
- Menyebut semua primary key sebagai clustered.
- Mengukur SELECT tetapi tidak mengukur INSERT/UPDATE.

PRAKTIK & EVIDENCE 2

Rancang dua candidate index untuk endpoint timeline. Bandingkan plan, logical reads/I/O, ukuran index, dan biaya insert.

Definition of done: Index dipilih berdasarkan workload; semantic query tidak berubah; read benefit dan write cost dicatat; index redundant dihindari.

CHECKPOINT Bab 2

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Mengapa satu table SQL Server hanya memiliki satu clustered index?

Apa arti left-prefix pada composite index?

Mengapa covering index tidak selalu gratis?

BAB 03

Aggregate reports yang benar dan transparan

Menyusun laporan kompleks tanpa double counting, ambiguous grain, atau query yang sulit diaudit.

Klasifikasi	CORE
Fokus silabus	More subquery — complex GROUP BY dengan aggregate serta clarity and transparency.
Project case	Laporan revenue membesar karena join fan-out dan aggregate dilakukan setelah tabel detail digabung tanpa menjaga grain.
Bukti Intermediate	Grain contract, reconciliation total, duplicate/null test, intermediate CTE/subquery result, dan query review notes.
Best practice	Nyatakan satu baris merepresentasikan apa; lakukan pre-aggregation pada grain yang tepat dan buat transformasi dapat diaudit.

TARGET KOMPETENSI

- Menggunakan GROUP BY dan aggregate secara benar.
- Mencegah fan-out double counting.
- Membuat query jelas melalui naming dan tahap logis.

3.1 Aggregate dan grain

Setiap selected column harus konsisten dengan grain hasil. COUNT(*) menghitung row; COUNT(column) mengabaikan NULL; COUNT(DISTINCT ...) mengubah semantic dan dapat mahal. Jangan memakai DISTINCT sebagai plester untuk join yang salah.

Contoh Bab 3. Query terarah

```
SELECT owner_id,
       COUNT(*) AS project_count,
       SUM(CASE WHEN status = 'late' THEN 1 ELSE 0 END) AS late_count
FROM projects
GROUP BY owner_id;
```

3.2 Fan-out dan pre-aggregation

Menggabungkan dua one-to-many relation sekaligus dapat menggandakan row. Agregasikan masing-masing child pada grain project, lalu join hasilnya. Uji dengan project yang memiliki banyak issue dan banyak time entry.

Contoh Bab 3. Query terarah

```
WITH issue_totals AS (
  SELECT project_id, COUNT(*) AS issue_count
  FROM issues GROUP BY project_id
), hour_totals AS (
  SELECT project_id, SUM(hours) AS total_hours
  FROM time_entries GROUP BY project_id
)
SELECT p.id, COALESCE(i.issue_count,0), COALESCE(h.total_hours,0)
FROM projects p
LEFT JOIN issue_totals i ON i.project_id = p.id
LEFT JOIN hour_totals h ON h.project_id = p.id;
```

3.3 Clarity dan transparency

Gunakan alias bermakna, satu grain per tahap, explicit columns, komentar untuk business rule yang tidak obvious, dan parameter. CTE bukan jaminan cepat; ia terutama alat struktur, dengan behavior optimasi bergantung DBMS/versi.

RED FLAG — KESALAHAN UMUM

- SELECT * pada laporan stabil.
- DISTINCT untuk menutupi duplicate dari join.
- COUNT(column) ketika NULL harus dihitung.
- Satu query sangat panjang tanpa tahap/grain yang jelas.

PRAKTIK & EVIDENCE 3

Bangun laporan per owner berisi project, issue, dan jam kerja tanpa double counting. Sertakan fixture pembuktian.

Definition of done: Satu row per owner; total cocok dengan kontrol; NULL/missing child tertangani; query dapat dijelaskan per tahap.

CHECKPOINT Bab 3

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa beda COUNT(*) dan COUNT(column)?

Mengapa dua relation one-to-many dapat menggandakan total?

Apakah CTE otomatis lebih cepat?

BAB 04

Self-join, hierarchy, dan pivot

Mengubah relasi baris menjadi struktur hierarchy dan laporan silang tanpa kehilangan semantic.

Klasifikasi	CORE
Fokus silabus	More subquery — self-join, pivots, dan conditional aggregation.
Project case	Hierarchy memiliki orphan atau cycle, sedangkan kategori pivot berubah dan menyebabkan kolom laporan tidak lengkap.
Bukti Intermediate	Hierarchy test data, orphan/cycle check, category inventory, pivot result, fallback conditional aggregation, dan deterministic order.
Best practice	Gunakan self-join hanya untuk kedalaman yang diketahui; validasi integritas hierarchy dan perlakukan pivot dinamis sebagai kontrak output.

TARGET KOMPETENSI

- Menggunakan self-join untuk relasi dalam table yang sama.
- Memilih conditional aggregation atau PIVOT.
- Menjaga kategori kosong dan duplicate tetap eksplisit.

4.1 Self-join

Self-join tepat saat satu table menyimpan relasi ke dirinya sendiri, misalnya employee-manager atau task-parent. Gunakan alias per peran dan LEFT JOIN jika root tanpa parent harus tetap muncul.

Contoh Bab 4. Query terarah

```
SELECT child.id, child.name,
       parent.id AS parent_id, parent.name AS parent_name
FROM tasks AS child
LEFT JOIN tasks AS parent ON parent.id = child.parent_id;
```


4.2 Conditional aggregation

Conditional aggregation lebih portabel daripada operator PIVOT dan sering lebih mudah dikontrol ketika kategori diketahui. Tentukan apakah nilai yang dibutuhkan count, sum, max, atau nilai terpilih.

Contoh Bab 4. Query terarah

```
SELECT owner_id,
       SUM(CASE WHEN status='active' THEN 1 ELSE 0 END) AS active_count,
       SUM(CASE WHEN status='late' THEN 1 ELSE 0 END) AS late_count
FROM projects
GROUP BY owner_id;
```

4.3 PIVOT dan kategori dinamis

SQL Server menyediakan PIVOT untuk memutar nilai row menjadi column. Daftar column biasanya harus diketahui saat query disusun; dynamic pivot memerlukan dynamic SQL yang tervalidasi dan menambah kompleksitas. Untuk kategori dinamis, pertimbangkan hasil row-based atau transformation pada layer report.

RED FLAG — KESALAHAN UMUM

- INNER self-join yang menghilangkan root.
- Pivot sebelum memastikan grain input.
- Dynamic SQL dari kategori user tanpa allowlist.
- Menganggap missing category sama dengan nilai nol.

PRAKTIK & EVIDENCE 4

Buat hierarchy task dan matriks status per owner menggunakan dua pendekatan. Bandingkan portability dan readability.

Definition of done: Root tetap tampil; kategori tidak menggandakan nilai; NULL/zero diputuskan sadar; pendekatan dipilih dengan alasan.

CHECKPOINT Bab 4

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Kapan self-join diperlukan?

Mengapa conditional aggregation lebih portabel?

Apa risiko dynamic pivot?

BAB 05

Subquery dan correlated reasoning

Memakai subquery untuk menyatakan set, scalar, dan korelasi tanpa perhitungan tersembunyi.

Klasifikasi	CORE
Fokus silabus	More subquery — single-table subquery, multiple-table subquery, dan alternative rewrite.
Project case	Correlated subquery dieksekusi berulang pada dataset besar dan menyamarkan hubungan cardinality yang sebenarnya dapat dibuat set-based.
Bukti Intermediate	Result-equivalence test, plan dan row count, logical reads, runtime representatif, serta rewrite comparison.
Best practice	Pilih subquery untuk mengekspresikan intent; ukur correlated execution dan refactor ketika join, aggregation, atau window lebih jelas serta efisien.

TARGET KOMPETENSI

- Membedakan scalar, table, dan correlated subquery.
- Menulis subquery lintas table yang benar.
- Mengevaluasi rewrite ke join atau window function.

5.1 Bentuk subquery

Scalar subquery harus menghasilkan paling banyak satu nilai; table subquery menghasilkan set; correlated subquery merujuk row outer dan secara logis dievaluasi per row, walau optimizer dapat mengubah rencana. Contract cardinality harus eksplisit.

Contoh Bab 5. Query terarah

```
SELECT p.id, p.name,
       (SELECT MAX(e.occurred_at)
        FROM project_events e
        WHERE e.project_id = p.id) AS last_event_at
FROM projects p;
```

5.2 Single-table dan multi-table

Subquery pada table sama dapat membandingkan row dengan aggregate kelompok. Multi-table subquery cocok untuk membership atau existence, tetapi join dapat lebih jelas saat column dari child diperlukan. Pilih bentuk yang paling jelas dan ukur plan.

Contoh Bab 5. Query terarah

```
SELECT p.id, p.budget
FROM projects p
WHERE p.budget > (
  SELECT AVG(p2.budget)
  FROM projects p2
  WHERE p2.department_id = p.department_id
);
```

5.3 Alternative rewrite

Window function dapat menghindari agregasi ulang saat membutuhkan detail dan aggregate kelompok dalam hasil yang sama. Rewrite hanya valid jika duplicate, NULL, tie, dan cardinality tetap sama.

RED FLAG — KESALAHAN UMUM

- Scalar subquery menghasilkan banyak row.
- Correlated subquery dipakai tanpa memeriksa plan.
- Rewrite join yang mengubah multiplicity.
- Mengasumsikan bentuk query menentukan urutan eksekusi fisik.

PRAKTIK & EVIDENCE 5

Tulis laporan project di atas rata-rata department menggunakan subquery dan window function; bandingkan hasil dan plan.

Definition of done: Cardinality contract terbukti; tie/NULL diuji; kedua versi semantic-equivalent; pilihan akhir didukung evidence.

CHECKPOINT Bab 5

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa contract scalar subquery?

Apakah correlated subquery selalu dijalankan literal per row?

Apa yang harus dijaga saat rewrite ke join?

BAB 06

EXISTS, IN, ANY, ALL, dan NULL semantics

Memilih operator set berdasarkan semantic, NULL behavior, dan kebutuhan hasil.

Klasifikasi	CORE
Fokus silabus	More subquery — EXISTS versus IN, ANY versus ALL, EXISTS, serta NOT EXISTS.
Project case	Anti-join memakai NOT IN terhadap himpunan yang mengandung NULL sehingga hasil menjadi kosong atau berbeda dari kebutuhan bisnis.
Bukti Intermediate	Truth-table NULL, empty-set test, duplicate test, expected result, plan comparison, dan semantic decision note.
Best practice	Pilih operator berdasarkan makna quantifier dan three-valued logic; gunakan NOT EXISTS untuk anti-semi join yang aman terhadap NULL.

TARGET KOMPETENSI

- Membedakan EXISTS dan IN.
- Menggunakan ANY dan ALL dengan comparator.
- Menghindari jebakan NOT IN dengan NULL.

6.1 EXISTS versus IN

EXISTS menanyakan apakah minimal satu row cocok dan tidak memerlukan column hasil; IN membandingkan nilai dengan sebuah set. Optimizer dapat menghasilkan plan serupa, tetapi semantic dan keterbacaan tetap penting. Gunakan correlated predicate yang lengkap pada EXISTS.

Contoh Bab 6. Query terarah

```
SELECT p.id, p.name
FROM projects p
WHERE EXISTS (
  SELECT 1 FROM issues i
  WHERE i.project_id = p.id AND i.severity = 'critical'
);
```

6.2 NOT EXISTS dan NULL

NOT IN dapat menghasilkan UNKNOWN bila subquery mengandung NULL, sehingga tidak ada row lolos seperti yang diperkirakan. NOT EXISTS biasanya lebih aman untuk anti-join karena korelasi menyatakan missing match secara langsung.

Contoh Bab 6. Query terarah

```
SELECT p.id
FROM projects p
WHERE NOT EXISTS (
  SELECT 1 FROM archived_projects a
  WHERE a.project_id = p.id
);
```

6.3 ANY dan ALL

`x > ANY(set)` berarti lebih besar dari setidaknya satu nilai; `x > ALL(set)` berarti lebih besar dari seluruh nilai. Periksa empty set dan NULL sesuai DBMS. MIN/MAX kadang lebih jelas, tetapi equivalence harus dibuktikan untuk NULL/empty set.

RED FLAG — KESALAHAN UMUM

- NOT IN pada nullable column.
- EXISTS tanpa correlation yang seharusnya ada.
- Menganggap IN selalu lambat dan EXISTS selalu cepat.
- Memakai ANY/ALL tanpa menjelaskan empty-set behavior.

PRAKTIK & EVIDENCE 6

Implementasikan inclusion dan exclusion project memakai IN/EXISTS/NOT EXISTS; tambahkan NULL dan empty-set tests.

Definition of done: Hasil sesuai truth table; NULL tidak menghilangkan row tak sengaja; plan dibandingkan; operator dipilih karena semantic.

CHECKPOINT Bab 6

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa perbedaan pertanyaan yang dijawab EXISTS dan IN?

Mengapa NOT IN berbahaya bila set mengandung NULL?

Apa arti > ALL?

BAB 07

Transactions, isolation, dan failure handling

Menempatkan atomicity, isolation, retry, dan error ownership pada boundary yang tepat.

Klasifikasi	CORE
Fokus silabus	Transaction — memahami transaksi, layering, dan penanganan failure ke lapisan pemanggil.
Project case	Order tersimpan tetapi inventory gagal, retry menduplikasi pembayaran, atau dua worker mengalami deadlock karena urutan lock berbeda.
Bukti Intermediate	Transaction boundary, rollback test, concurrency timeline, isolation observation, deadlock/retry log, dan idempotency proof.
Best practice	Jaga transaksi singkat dan boundary eksplisit; propagasikan failure, gunakan retry terbatas hanya untuk error transient dan operasi idempotent.

TARGET KOMPETENSI

- Mendesain transaction boundary pendek dan kohesif.
- Menjelaskan isolation anomaly.
- Menangani deadlock/serialization failure sampai application layer.

7.1 ACID dan boundary

Transaction membungkus perubahan yang harus commit atau rollback bersama. Jangan mempertahankan lock saat menunggu user, file besar, atau network call. Validasi yang tidak membutuhkan lock sebaiknya dilakukan sebelum transaction; invariant kritis tetap ditegakkan constraint/locking di database.

Contoh Bab 7. Query terarah

```
BEGIN;
UPDATE projects
SET budget = budget - :amount
WHERE id = :source_id AND budget >= :amount;

UPDATE projects
SET budget = budget + :amount
WHERE id = :target_id;
COMMIT;
```

7.2 Isolation dan concurrency

Dirty read, non-repeatable read, phantom, lost update, dan write skew bergantung isolation serta pattern akses. Nama level sama tidak selalu identik antar-DBMS. Gunakan constraint, atomic update, row lock, optimistic version, atau serializable sesuai invariant.

Contoh Bab 7. Query terarah

```
UPDATE projects
SET status = :new_status, version = version + 1
WHERE id = :id AND version = :expected_version;
-- Affected rows = 0 means conflict or missing row.
```

7.3 Failure propagation dan retry

Database failure harus diterjemahkan tanpa ditelan. Deadlock/serialization failure dapat di-retry pada boundary yang idempotent dengan batas dan backoff. Jangan retry semua error. Layer atas perlu menerima status yang cukup untuk memutuskan rollback, retry, atau respons pengguna.

RED FLAG — KESALAHAN UMUM

- Transaction mencakup HTTP call.
- Catch error lalu commit sebagian.
- Read-modify-write tanpa concurrency control.
- Retry tanpa idempotency dan batas percobaan.

PRAKTIK & EVIDENCE 7

Simulasikan dua session memperbarui project yang sama. Buktikan lost update, lalu perbaiki dengan optimistic version atau locking.

Definition of done: Invariant terjaga; transaction pendek; konflik terdeteksi; error sampai boundary aplikasi; retry hanya untuk failure yang tepat.

CHECKPOINT Bab 7

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Mengapa transaction harus pendek?

Apa fungsi version column?

Apakah semua database error boleh di-retry?

BAB 08

Views, layered views, dan stored procedures

Mengemas query reusable tanpa menyembunyikan dependency, security, atau performance cost.

Klasifikasi	CORE
Fokus silabus	Database Objects — views, layered views, top-value views, dan stored procedure parameter queries.
Project case	Layered view menyembunyikan join mahal, TOP tidak memiliki urutan deterministik, dan procedure memberi hasil berbeda karena parameter sensitivity.
Bukti Intermediate	Dependency map, view/procedure definition, permission test, deterministic-order test, plan per parameter, dan contract result.
Best practice	Gunakan database object sebagai boundary yang jelas; batasi layer, nyatakan urutan, dan ukur plan pada parameter representatif.

TARGET KOMPETENSI

- Mendesain view dan layered view.
- Membuat top-N view dengan tie rule jelas.
- Membedakan view dari parameterized procedure/function.

8.1 Views

View memberi nama pada query dan dapat menjadi abstraction/security surface. View biasa umumnya tidak menyimpan hasil; query consumer tetap dioptimasi terhadap underlying objects. Jangan menganggap view sebagai cache atau otomatis cepat.

Contoh Bab 8. Query terarah

```
CREATE VIEW reporting.v_project_health AS
SELECT p.id, p.owner_id, p.status,
       COUNT(i.id) AS issue_count
FROM projects p
LEFT JOIN issues i ON i.project_id = p.id
GROUP BY p.id, p.owner_id, p.status;
```

8.2 Layered dan top-N views

Layered view dapat meningkatkan reuse, tetapi rantai panjang menyulitkan lineage dan tuning. Top-N membutuhkan ORDER BY, tie policy, dan partition scope yang jelas. Banyak DBMS tidak menjamin order saat SELECT dari view tanpa ORDER BY pada query consumer.

8.3 Parameters dan stored procedures

View biasa tidak menerima parameter pada DBMS umum. Parameterized query ditempatkan pada prepared statement, table-valued function, atau stored procedure sesuai engine. Procedure berguna untuk operasi dekat data, permission boundary, atau batch terkendali; versioning dan observability tetap diperlukan.

Contoh Bab 8. Query terarah

```
-- SQL Server example
CREATE PROCEDURE reporting.GetProjectSummary
    @TenantId bigint,
    @FromDate date
AS
BEGIN
    SET NOCOUNT ON;
    SELECT project_id, COUNT(*) AS event_count
    FROM dbo.project_events
    WHERE tenant_id = @TenantId AND occurred_at >= @FromDate
    GROUP BY project_id;
END;
```

RED FLAG — KESALAHAN UMUM

- Menganggap view menerima parameter.
- View berlapis tanpa lineage.
- TOP/LIMIT tanpa ordering deterministik.
- Procedure menjadi tempat seluruh business logic tanpa test/versioning.

PRAKTIK & EVIDENCE 8

Bangun base view dan top-project report. Buat dependency map dan satu parameterized procedure/function sesuai DBMS lab.

Definition of done: Grain view jelas; top-N deterministik; parameter bound; dependency dan permission terdokumentasi; plan consumer diperiksa.

CHECKPOINT Bab 8

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apakah view biasa menyimpan hasil?

Mengapa top-N membutuhkan tie policy?

Bisakah view biasa menerima parameter?

BAB 09

Triggers: power, risk, dan alternatives

Menggunakan trigger hanya ketika invariant dekat data lebih kuat daripada biaya side effect tersembunyi.

Klasifikasi	CORE
Fokus silabus	Trigger — mengetahui cara menggunakan trigger dan kapan tidak menggunakannya.
Project case	Trigger diasumsikan berjalan per baris, gagal saat bulk update, memicu recursion, atau menyembunyikan side effect dari service.
Bukti Intermediate	Multi-row test, transaction/rollback behavior, recursion guard, latency metric, audit requirement, dan alternative decision.
Best practice	Gunakan trigger untuk invariant atau audit yang benar-benar dekat data; desain set-based, kecil, observable, dan terdokumentasi.

TARGET KOMPETENSI

- Menjelaskan timing dan row/statement scope.
- Menjaga trigger set-based dan aman pada multi-row DML.
- Memilih alternatives ketika trigger tidak tepat.

9.1 Kapan trigger layak

Trigger dapat cocok untuk audit minimum, derived invariant, atau integrasi legacy yang harus berlaku pada semua writer. Namun trigger menyembunyikan side effect dari caller, menambah lock/work, dan dapat memicu recursion/cascade. Constraint sering lebih baik untuk aturan declarative.

9.2 Multi-row correctness

Jangan mengasumsikan satu row per statement. SQL Server `inserted`/`deleted` dan transition tables/row triggers pada engine lain memiliki model berbeda. Trigger harus diuji dengan bulk insert/update, rollback, error, dan recursion.

Contoh Bab 9. Query terarah

```
-- SQL Server example: set-based audit
INSERT INTO audit.project_status_changes(project_id, old_status, new_status, changed_at)
SELECT i.id, d.status, i.status, SYSUTCDATETIME()
FROM inserted i
JOIN deleted d ON d.id = i.id
WHERE i.status <> d.status;
```

9.3 Alternatives

Gunakan constraint untuk invariant declarative, explicit application service untuk workflow yang perlu orchestration, event/outbox untuk integrasi asynchronous, dan scheduled job untuk rekonsiliasi. Pilihan ditentukan oleh ownership dan failure semantics.

RED FLAG — KESALAHAN UMUM

- Trigger melakukan network call.
- Logic hanya benar untuk satu row.
- Recursive trigger tanpa guard.
- Audit table menyimpan secret/PII berlebihan.

PRAKTIK & EVIDENCE 9

Review satu kebutuhan audit/status. Implementasikan trigger set-based atau tulis ADR mengapa alternative lebih tepat.

Definition of done: Pilihan dapat dijelaskan; multi-row dan rollback teruji; side effect/log aman; recursion dan deployment order dikendalikan.

CHECKPOINT Bab 9

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa risiko utama trigger?

Mengapa trigger harus diuji dengan multi-row DML?

Kapan constraint lebih tepat?

BAB 10

OLTP/OLAP, access control, log, dan profiling

Memisahkan workload transaksi dan analitik serta mengamati database dengan akses minimum.

Klasifikasi	CORE
Fokus silabus	User Access Control — profiling/log untuk monitoring serta perbedaan OLTP dan OLAP.
Project case	Reporting scan membebani database transaksi, akun aplikasi memiliki privilege berlebih, dan log query membocorkan parameter sensitif.
Bukti Intermediate	Workload classification, least-privilege matrix, sanitized log, wait/lock/profile snapshot, dan monitoring hypothesis.
Best practice	Pisahkan karakter workload, gunakan least privilege, dan lakukan profiling terarah dengan data sensitif dimasking serta overhead terukur.

TARGET KOMPETENSI

- Membedakan OLTP dan OLAP.
- Menerapkan least privilege.
- Membaca slow-query/log/profiling dengan aman.

10.1 OLTP versus OLAP

OLTP melayani transaksi pendek, concurrency tinggi, dan data operasional mutakhir. OLAP melayani scan/agregasi besar, history, dan model dimensional. Reporting berat dapat dipindahkan ke read replica, warehouse, summary table/materialized view, atau pipeline terjadwal sesuai freshness dan consistency requirement.

10.2 User access control

Pisahkan identity aplikasi, migration, reporting, monitoring, dan admin. Berikan privilege minimum pada object yang diperlukan; hindari shared superuser. Rotation,

expiry, network boundary, row-level policy, dan audit harus mengikuti capability DBMS serta kebijakan perusahaan.

Contoh Bab 10. Query terarah

```
-- Conceptual; adapt to the DBMS
GRANT SELECT ON reporting.v_project_health TO reporting_reader;
GRANT SELECT, INSERT, UPDATE ON app.projects TO application_runtime;
-- Do not grant schema-wide write access without need.
```

10.3 Profiling dan log

Monitoring user cukup membaca metric/log/plan yang dibutuhkan tanpa melihat password atau payload sensitif. Catat query fingerprint, duration, rows, waits/locks, I/O, error, dan correlation ID. Parameter/PII perlu redaction; akses production harus terkontrol dan auditable.

RED FLAG — KESALAHAN UMUM

- Dashboard berat dijalankan pada primary saat jam sibuk.
- Aplikasi memakai account admin.
- Log menyimpan literal password/token.
- Menyalin data production ke laptop tanpa kontrol.

PRAKTIK & EVIDENCE 10

Klasifikasikan lima query sebagai OLTP/OLAP, usulkan placement, lalu rancang role matrix runtime/reporting/monitoring/migration.

Definition of done: Workload dan freshness jelas; privilege minimum; secret teredaksi; akses monitoring tidak dapat mengubah business data.

CHECKPOINT Bab 10

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa pembeda utama OLTP dan OLAP?

Mengapa monitoring user tidak perlu write access?

Apa yang sebaiknya direkam dari slow query?

BAB 11

Execution plans dan evidence-based tuning

Membaca execution plan sebagai evidence, bukan sebagai daftar ikon untuk dihafal.

Klasifikasi	CORE
Fokus silabus	Tuning — membaca execution plan dan memahami pengaruh bagian query terhadap rencana eksekusi.
Project case	Tim menambah index dari asumsi; akar masalah sebenarnya adalah cardinality estimate, implicit conversion, stale statistics, atau memory spill.
Bukti Intermediate	Plan actual/estimated, row estimate gap, operator cost symptom, I/O/time, wait/spill, statistics state, dan before-after metric.
Best practice	Tuning adalah siklus hypothesis–evidence–change–verification; optimasi tidak boleh mengubah semantik atau hanya menang pada cache hangat.

TARGET KOMPETENSI

- Membedakan estimated dan actual plan.
- Membaca row estimate, access, join, sort/hash, dan I/O.
- Menjalankan tuning loop yang terkontrol.

11.1 Plan dan risiko eksekusi

Estimated plan tidak menjalankan query; actual plan menambah runtime statistics dan dapat mengeksekusi query. `EXPLAIN ANALYZE` pada sebagian DBMS benar-benar menjalankan statement, sehingga gunakan transaction/read-only, dataset aman, dan izin yang tepat. Jangan menjalankannya sembarang pada DML production.

Contoh Bab 11. Query terarah

```
-- PostgreSQL example; executes the SELECT
EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT project_id, COUNT(*)
FROM project_events
WHERE tenant_id = 42
      AND occurred_at >= DATE '2026-01-01'
GROUP BY project_id;
```

11.2 Membaca plan

Mulai dari actual versus estimated rows, node dengan work/I/O tinggi, access path, join algorithm, sort/hash spill, filter rows removed, lookup, parallelism, dan waits.

Misestimate dapat berasal dari statistik stale, correlation antar-column, parameter sensitivity, atau data skew.

11.3 Tuning loop

Bekukan correctness test dan baseline; ubah satu hal; ukur pada parameter/data representatif; periksa efek read dan write; simpan before–after plan; siapkan rollback. Query time tunggal tanpa context cache/concurrency bukan bukti kuat.

Contoh Bab 11. Query terarah

```
-- Keep the predicate typed and explicit
SELECT id, name
FROM projects
WHERE tenant_id = :tenant_id
AND status = :status
ORDER BY updated_at DESC
FETCH FIRST 50 ROWS ONLY;
```

RED FLAG — KESALAHAN UMUM

- Melihat cost sebagai millisecond universal.
- Mengabaikan estimate-vs-actual gap.
- Menambah index tanpa menguji write path.
- Membandingkan run pertama dan kedua tanpa mencatat cache.

PRAKTIK & EVIDENCE 11

Ambil satu slow query, tulis hypothesis, simpan plan/metric baseline, lakukan satu perubahan, lalu ukur ulang.

Definition of done: Result identik; environment dicatat; evidence before–after tersimpan; regression/read-write impact dan rollback jelas.

CHECKPOINT Bab 11

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Apa beda estimated dan actual plan?

Apakah cost plan sama dengan millisecond?

Mengapa estimate-vs-actual penting?

BAB 12

N+1 refactoring dan mini-project

Menghilangkan N+1 dan mengintegrasikan seluruh kompetensi pada kasus database project.

Klasifikasi	CORE — INTEGRATED VALIDATION
Fokus silabus	Query Refactoring — mendeteksi N+1 dan mengoptimasi melalui eager loading, joins, batching, serta index yang tepat.
Project case	Endpoint cepat pada data kecil tetapi menghasilkan ratusan query ketika daftar membesar; perbaikan join tunggal lalu menggandakan hasil.
Bukti Intermediate	Query-count regression, trace request, result-equivalence test, latency/I/O baseline, selected loading strategy, index proof, dan rollback.
Best practice	Ukur interaksi database per use case; refactor bertahap, kunci correctness, lalu validasi query count, resource, dan maintainability.

TARGET KOMPETENSI

- Mendeteksi N+1 dari log/query count.
- Memilih eager loading, join, aggregate, atau batching.
- Menyelesaikan Project Delivery Analytics Database.

12.1 N+1 sebagai interaction smell

N+1 terjadi ketika satu query parent diikuti query child per row, sering melalui lazy loading dalam loop. Buktikan dengan query log/APM dan relation access. Perbaikan dapat berupa eager loading, join, aggregate subquery, data loader, atau batching; pilihan bergantung shape hasil dan memory.

Contoh Bab 12. Query terarah

```
-- Set-based summary instead of one query per project
SELECT p.id, p.name, COUNT(i.id) AS open_issue_count
FROM projects p
LEFT JOIN issues i
  ON i.project_id = p.id AND i.status = 'open'
WHERE p.tenant_id = :tenant_id
GROUP BY p.id, p.name;
```

12.2 Refactoring sequence

Kunci output dengan characterization test, ukur query count dan latency, perbaiki access pattern, tambahkan index hanya bila plan/workload mendukung, lalu uji authorization dan pagination. Eager loading semua relation dapat memindahkan masalah menjadi over-fetching dan memory spike.

12.3 Mini-project — Project Delivery Analytics Database

Bangun schema dan query pack untuk project, task hierarchy, issue, event, dan time entry. Sediakan OLTP operations, laporan analitik, view/procedure, transaction case, role matrix, serta tuning evidence. Assessment final akan memakai schema dan kasus baru agar mengukur transfer kompetensi, bukan hafalan project.

RED FLAG — KESALAHAN UMUM

- Menilai N+1 hanya dari waktu lokal.
- Eager-load semua relation.
- Join mengubah pagination/multiplicity.
- Optimasi mengabaikan tenant/authorization predicate.

PRAKTIK & EVIDENCE 12

Selesaikan mini-project, lakukan peer review, perbaiki minimal dua temuan, dan presentasikan correctness, concurrency, security, serta plan before–after.

Definition of done: Schema reproducible; query correctness teruji; N+1 hilang; transaction invariant aman; least privilege; plan/metric dan rollback terdokumentasi.

CHECKPOINT Bab 12

Petunjuk: Jawab tanpa melihat materi selama 3–5 menit. Cocokkan jawaban pada Lampiran C setelah selesai.

Bagaimana membuktikan N+1?

Mengapa eager loading bukan jawaban universal?

Apa beda mini-project dan assessment final?

Lampiran A1 — Traceability Detail 21 Topik Silabus

AUDIT CAKUPAN			
Tabel berikut memetakan seluruh 21 entri topik SQL pada sheet Silabus Intermediate Programmer. Nomor mengikuti urutan topik pada domain SQL, bukan nomor soal.			
No.	Kategori	Topik silabus	Bab
1	Indexing	How indexes work, difference between clustered, non-clustered, etc, what a page is and how they layout	1–2
2	More subquery	Pivot queries	4
3	More subquery	Self-join when relevant	4
4	More subquery	Complex reports with GROUP BY and aggregate functions	3
5	More subquery	Clarity and Transparency	3
6	More subquery	Single Table Sub Query	5
7	More subquery	Multiple Table Sub Query	5
8	More subquery	EXISTS versus IN operators	6
9	More subquery	ANY versus ALL operators	6
10	More subquery	Using the EXISTS operator	6
11	More subquery	Using the NOT EXISTS operator	6

No.	Kategori	Topik silabus	Bab
12	User Access Control	Basic profiling for monitoring/debugging, including log reading	10
13	User Access Control	Understands the difference between OLAP and OLTP and when/where to use OLAP structures	10
14	Trigger	When to use and avoid triggers	9
15	Transaction	Understands transactions and can layer them handling failures up the stack	7
16	Database Objects	Views	8
17	Database Objects	Layered Views	8
18	Database Objects	Create Top Value Views	8
19	Database Objects	Stored procedure parameter Queries using Variables through Views	8
20	Tuning	Read execution plans and understand how query parts affect them	11
21	Query Refactoring	N+1 Problem & Indexing	12

Lampiran A2 — Traceability Kelompok Silabus ke Modul

Kelompok silabus	Bab	Bukti
Indexing: pages, clustered/nonclustered, access path	1–2	Prediction, plan, read/write trade-off
Pivot, self-join, aggregate reports, clarity	3–4	Result-set and edge-case tests
Single/multiple subquery	5	Cardinality and rewrite evidence
EXISTS/IN, ANY/ALL, NOT EXISTS	6	NULL and empty-set tests
Transaction and layered failure handling	7	Multi-session concurrency evidence
Views, layered/top views, parameterized procedure	8	Dependency, permission, and plan
Triggers: use and avoid	9	Multi-row and rollback tests
Profiling/log, OLTP/OLAP, user access	10	Role matrix and safe monitoring
Execution plan and tuning	11	Before–after plan and metric
N+1 and indexing refactor	12	Query-count and regression evidence

Lampiran B — Rubrik Self-Check Intermediate

Dimensi	Belum memenuhi	Memenuhi	Kuat
Correctness	Happy path; grain kabur	Grain dan edge case teruji	Invariant/multiplicity dapat dibuktikan
Readability	Alias/logic sulit ditelusuri	Tahap dan naming jelas	Query menjelaskan business rule
Concurrency	Transaction asal membungkus	Boundary/isolation tepat	Conflict/retry teruji multi-session
Security	Shared admin account	Least privilege dan binding	Audit/redaction/tenant boundary kuat
Performance	Menebak dari durasi	Plan dan baseline digunakan	Read/write/concurrency trade-off terukur
Decision	Index/pattern tanpa alasan	Pilihan sesuai workload	Alternative dan rollback eksplisit

Interpretasi: Siap assessment bila seluruh dimensi minimal Memenuhi dan tidak ada blocker pada data integrity, authorization/tenant isolation, transaction safety, reproducibility, atau correctness laporan.

Lampiran C — Kunci Quick Check

Nomor	Jawaban inti
1.1	I/O dilakukan per block/page sehingga satu lookup dapat membaca lebih dari satu nilai.
1.2	Tidak; scan dapat optimal saat porsi row besar.
1.3	Menetapkan satu row mewakili apa.
2.1	Leaf clustered memuat row data; organisasi fisik hanya satu.
2.2	Urutan key menentukan prefix akses.
2.3	Menambah storage dan biaya DML.
3.1	COUNT(*) semua row; COUNT(column) mengabaikan NULL.
3.2	Fan-out menghasilkan kombinasi child.
3.3	Tidak; terutama membantu struktur dan bergantung optimizer.
4.1	Relasi dua peran pada table yang sama.
4.2	Menggunakan CASE dan aggregate portabel.
4.3	Injection, kompleksitas, dan schema hasil berubah.
5.1	Maksimal satu nilai/row sesuai context.
5.2	Tidak selalu; optimizer dapat decorrelate/rewrite.
5.3	Multiplicity, NULL, tie, dan hasil.
6.1	EXISTS menguji keberadaan; IN membandingkan nilai dengan set.
6.2	NULL membuat hasil UNKNOWN.
6.3	Lebih besar dari setiap nilai dalam set.
7.1	Mengurangi lock duration dan contention.

Nomor	Jawaban inti
7.2	Mendeteksi perubahan sejak read.
7.3	Tidak; hanya failure transient tertentu.
8.1	Tidak, kecuali materialized/indexed variant tertentu.
8.2	Hasil harus deterministik saat tie.
8.3	Umumnya tidak; gunakan function/procedure/prepared query.
9.1	Side effect tersembunyi, lock, recursion, dan debugging.
9.2	Satu statement dapat memengaruhi banyak row.
9.3	Untuk invariant declarative.
10.1	OLTP transaksi pendek; OLAP scan/agregasi/history.
10.2	Observasi tidak membutuhkan mutasi data.
10.3	Fingerprint, duration, rows, I/O, waits, error, correlation.
11.1	Actual menjalankan/menyertakan runtime statistics.
11.2	Tidak; unit biaya internal optimizer.
11.3	Menunjukkan misestimate yang memengaruhi plan.
12.1	Query log/APM menunjukkan satu parent plus query per row.
12.2	Dapat over-fetch dan menaikkan memory.
12.3	Project melatih; assessment memakai kasus baru.

Lampiran D — Query Investigation & Tuning Log

Field	Catatan
Business question/grain	Satu row mewakili apa dan siapa consumer-nya
Environment	DBMS/version, schema, index, data volume/skew, cache/concurrency
Query/parameters	Fingerprint dan parameter representatif; redact secret/PII
Correctness controls	Expected result, NULL, duplicate, missing relation, tie

Field	Catatan
Baseline	Duration, rows, I/O/pages, query count, locks/waits, CPU/memory
Plan evidence	Estimate/actual rows, access, join, sort/hash, spill/lookup
Hypothesis	Mekanisme penyebab yang dapat diuji
Change	Satu perubahan pada query/index/statistics/schema/access pattern
Before–after	Metric dan plan; correctness tetap sama
Write/concurrency impact	DML cost, lock, storage, production risk
Rollback	Cara mengembalikan dan trigger pemulihan

Lampiran E — Glosarium

Istilah	Definisi
Grain	Makna satu row pada dataset atau hasil query.
Cardinality	Jumlah row atau jumlah nilai berbeda pada konteks tertentu.
Page/block	Unit penyimpanan/I/O database.
Sargable	Predicate yang dapat mendukung access path index tertentu.
Covering index	Index yang menyediakan kolom query tanpa lookup tambahan tertentu.
Fan-out	Peningkatan row akibat join relasi one-to-many/many-to-many.
Isolation	Aturan visibility dan interaksi antar-transaction.
Execution plan	Rencana operator fisik/logis yang dipilih optimizer.
N+1	Satu query awal diikuti query per row/relation.
OLTP/OLAP	Workload transaksi operasional / workload analitik.

Lampiran F — Referensi Resmi

Gunakan dokumentasi sesuai DBMS dan versi project. Tanggal akses: 20 Agustus 2026.

- [PostgreSQL — Indexes](#)
- [PostgreSQL — Using EXPLAIN](#)
- [PostgreSQL — Transaction Isolation](#)
- [PostgreSQL — CREATE VIEW](#)
- [PostgreSQL — Trigger Behavior](#)
- [SQL Server — Clustered and Nonclustered Indexes](#)
- [SQL Server — Index Architecture and Design Guide](#)
- [SQL Server — PIVOT and UNPIVOT](#)
- [SQL Server — Display an Actual Execution Plan](#)
- [MySQL — Optimization and Indexes](#)
- [MySQL — EXPLAIN](#)
- [MySQL — InnoDB Transaction Model](#)